Standard SQL Functions (PostgreSQL, Oracle, MySQL 8+, Snowflake)

Cross-RDBMS

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
`REGEXP_LIKE(str, pattern [, flags])`	Returns TRUE/1 if regular expression matches anywhere in str; supported in Oracle, MySQL 8+, Snowflake, Databricks **EXAMPLE:** `SELECT * FROM users WHERE REGEXP_LIKE(email, '^[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}$', 'i');`
`REGEXP_SUBSTR(str, pattern [, pos [, occ [, flags [, group]]]])`	Extracts the matched substring (or specific capture group) from str; pos defaults to 1, occ defaults to 1 **EXAMPLE:** `SELECT REGEXP_SUBSTR('invoice_2026_08.pdf', '\d{4}') → '2026'`
`REGEXP_REPLACE(str, pattern, replacement [, pos [, occ [, flags]]])`	Replaces matched substrings in str with replacement text (supports backreferences \1, \2 or $1, $2) **EXAMPLE:** `SELECT REGEXP_REPLACE('phone: 555-1234', '\d', 'X') → 'phone: XXX-XXXX'`
`REGEXP_INSTR(str, pattern [, pos [, occ [, return_opt [, flags]]]])`	Returns 1-based start position of the N-th match (or position after match if return_opt = 1); 0 if no match **EXAMPLE:** `SELECT REGEXP_INSTR('Order #4092 created', '\d+') → 8`
`REGEXP_COUNT(str, pattern [, pos [, flags]])`	Returns total count of non-overlapping matches in str (PostgreSQL 15+, Oracle, Snowflake, MySQL 8+) **EXAMPLE:** `SELECT REGEXP_COUNT('cat, dog, bird, fish', '\w+') → 4`

PostgreSQL Regex Operators & Native Functions

Cross-RDBMS

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
`str ~ pattern`	POSIX regular expression match operator (case-sensitive) — returns boolean true/false EXAMPLE: `SELECT '2026-08' ~ '^\d{4}-\d{2}$' → true`
`str ~* pattern`	POSIX regular expression match operator (case-insensitive) — returns boolean true/false EXAMPLE: `SELECT 'Alice' ~* '^a.*e$' → true`
`str !~ pattern / str !~* pattern`	POSIX regular expression non-match operators (case-sensitive !~ and case-insensitive !~*) EXAMPLE: `SELECT 'hello' !~* '\d' → true`
`REGEXP_MATCH(str, pattern [, flags])`	Extracts first match as a text[] array of captured groups (or entire match if no groups); returns NULL if no match EXAMPLE: `SELECT (REGEXP_MATCH('id: 420', 'id: (\d+)'))[1] → '420'`
`REGEXP_MATCHES(str, pattern [, flags])`	Returns set of text[] arrays for all matches (use 'g' flag for global multi-row set returning function) EXAMPLE: `SELECT REGEXP_MATCHES('a1 b2 c3', '([a-z])(\d)', 'g');`
`REGEXP_SPLIT_TO_TABLE(str, pattern [, flags])`	Splits string by regex matches and returns each item as a separate table row (set of text) EXAMPLE: `SELECT REGEXP_SPLIT_TO_TABLE('apple,banana;cherry', '[,;]\s*');`
`REGEXP_SPLIT_TO_ARRAY(str, pattern [, flags])`	Splits string by regex matches into a single PostgreSQL array (text[]) EXAMPLE: `SELECT REGEXP_SPLIT_TO_ARRAY('10.20.30', '\.') → '{10,20,30}'`
`str SIMILAR TO pattern`	SQL standard pattern matching using SQL99 rules (combines LIKE wildcards % _ with regex | * + ( ) EXAMPLE: `SELECT 'abc' SIMILAR TO '%(b|d)%' → true`

MySQL, MariaDB, & SQLite

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
`str REGEXP pattern / str RLIKE pattern`	Matches regular expression anywhere within str; RLIKE is an exact synonym of REGEXP in MySQL and MariaDB **EXAMPLE:** `SELECT 'Product 99' REGEXP '[0-9]+' → 1`
`str NOT REGEXP pattern / str NOT RLIKE pattern`	Returns 1 if pattern does not match str, 0 if it matches, and NULL if either argument is NULL **EXAMPLE:** `SELECT 'abc' NOT REGEXP '\d' → 1`
`REGEXP_LIKE(expr, pat [, match_type])`	MySQL 8.0+ functional syntax with match_type flags ('i', 'c', 'm', 'n', 'u') **EXAMPLE:** `SELECT REGEXP_LIKE('Admin_User', '^admin', 'i') → 1`
`REGEXP_SUBSTR(expr, pat [, pos [, occurrence [, match_type]]])`	MySQL 8.0+ function returning matched substring, or NULL if no match found **EXAMPLE:** `SELECT REGEXP_SUBSTR('item #429 in stock', '\d+') → '429'`
`str REGEXP pattern (SQLite)`	SQLite regex operator — requires the regexp() extension function to be loaded/registered by the host driver **EXAMPLE:** `SELECT * FROM items WHERE name REGEXP '^A[0-9]+';`

Cloud Data Warehouses (Snowflake, BigQuery, Databricks, Trino)

Cross-RDBMS

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
`REGEXP_CONTAINS(value, regex) (BigQuery)`	Returns TRUE if regex finds a match anywhere in string value in Google BigQuery EXAMPLE: `SELECT REGEXP_CONTAINS(url, r'https?://[a-z0-9.-]+') FROM web_traffic;`
`REGEXP_EXTRACT(value, regex [, position [, occurrence]]) (BigQuery)`	Extracts the first matching substring or first capturing group from value in BigQuery EXAMPLE: `SELECT REGEXP_EXTRACT('order_id: 9941', r'\d+') → '9941'`
`REGEXP_EXTRACT_ALL(value, regex) (BigQuery / Trino)`	Returns an ARRAY of all non-overlapping matches or capture groups in BigQuery and Trino/Presto EXAMPLE: `SELECT REGEXP_EXTRACT_ALL('v1.0, v2.1, v3.5', r'v\d+\.\d+') → ['v1.0', 'v2.1', 'v3.5']`
`REGEXP_LIKE(str, pattern [, parameters]) (Snowflake)`	Evaluates regex match against string column with parameter flags in Snowflake / Databricks EXAMPLE: `SELECT * FROM sales WHERE REGEXP_LIKE(sku, '^[A-Z]{3}-\d{4}$');`
`str RLIKE pattern (Spark / Databricks / Hive / Trino)`	Tests if str matches pattern regular expression using Java regex syntax under the hood EXAMPLE: `SELECT * FROM events WHERE event_name RLIKE '^user_(login|logout)$';`

SQL Match Parameters & Flag Modifiers

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
`'i'`	Case-insensitive matching parameter (e.g. REGEXP_LIKE(name, '^john', 'i')) **EXAMPLE:** `REGEXP_LIKE('JOHN', 'john', 'i') → TRUE`
`'c'`	Case-sensitive matching parameter (default in most SQL engines except case-insensitive collations) **EXAMPLE:** `REGEXP_LIKE('JOHN', 'john', 'c') → FALSE`
`'m'`	Multiline mode — anchors ^ and $ match beginning and end of each physical line within string **EXAMPLE:** `REGEXP_REPLACE(multiline_text, '^#\s*', '', 1, 0, 'm')`
`'n' / 's'`	Singleline / dot-all mode — permits dot (.) to match newline characters (flags vary: 'n' in Oracle/Postgres, 's' in PCRE) **EXAMPLE:** `REGEXP_SUBSTR(html_doc, '<div>.*</div>', 1, 1, 'n')`
`'x'`	Extended/verbose mode — ignores whitespace and allows # comments inside regex pattern string **EXAMPLE:** `REGEXP_LIKE(val, '^\d{4} # Year \n-\d{2} # Month', 'x')`
`'g' (PostgreSQL)`	Global replacement / match parameter in PostgreSQL (replaces or matches all occurrences) **EXAMPLE:** `REGEXP_REPLACE('a1 b2 c3', '\d', 'X', 'g') → 'aX bX cX'`

SQL Server (T-SQL) LIKE Character Ranges & Wildcards

Cross-RDBMS

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
`LIKE '%[0-9]%'`	Matches any string containing at least one digit character (0 through 9) in T-SQL EXAMPLE: `SELECT * FROM orders WHERE order_num LIKE 'ORD-[0-9][0-9][0-9][0-9]';`
`LIKE '%[^0-9]%'`	Negated set — matches any string containing at least one character that is NOT a digit EXAMPLE: `SELECT * FROM users WHERE zip_code LIKE '%[^0-9]%';`
`LIKE '[A-Za-z]%'`	Matches strings starting with any alphabetic letter (A-Z or a-z) EXAMPLE: `SELECT * FROM customers WHERE name LIKE '[A-M]%';`
`PATINDEX('%[0-9]%', str)`	Returns 1-based start position of first occurrence of pattern in str; returns 0 if not found EXAMPLE: `SELECT PATINDEX('%[0-9]%', 'Item #42') → 7`
`LIKE '%[%_]%' (ESCAPE '\')`	Escapes literal % or _ wildcard characters using custom ESCAPE clause EXAMPLE: `SELECT * FROM logs WHERE message LIKE '%100\%%' ESCAPE '\';`